El pulsador: digitalRead e if en Arduino

Lección 11 — 1MAK, SST-INF-01c — Repaso del if, leer un pulsador, el circuito en Tinkercad, errores frecuentes, ejercicio con cuatro LED

13/11/2026

1 Desde dónde empezamos: el if

En la lección anterior (A10) le dimos a Arduino la posibilidad de **tomar decisiones**. La herramienta es la instrucción `if`, que en inglés quiere decir "**si**": *si* pasa algo, *entonces* ejecuta otra cosa.

Definición

Estructura del if. Se escribe `if`, después entre **paréntesis** (*parentesi tonde*) la **condición** (*condizione*); debajo de la "i" se abre la **llave** (*graffa*), y más abajo la llave se cierra. La llave abierta y la llave cerrada **delimitan** una parte del código, es decir un grupo de **instrucciones** (órdenes) que le damos a la máquina.
La **condición** es una expresión que puede dar solo dos resultados: **verdadero** o **falso** (*vero*, *falso*).

- Si es **verdadera**, Arduino **ejecuta** el código entre las llaves (por ejemplo un `digitalWrite`, o todo lo que vamos a aprender en las próximas semanas).
- Si es **falsa**, lo **salta por completo**: salta toda esa parte del código y sigue con lo que está escrito después.

```
if (condizione)
{
  // aquí el código que se ejecuta si la condición es verdadera
}
```

1.1 Un ejemplo con dos variables

Una condición puede ser `a > b` ("**a mayor que b**"). Tenemos dos variables: `a` vale **5** y `b` vale **3**.

- ¿5 es mayor que 3? **Sí**: la condición es verdadera y Arduino **ejecuta** el código entre las llaves.
- Si por algún motivo la variable cambia y `a`, en lugar de 5, vale **1**: 1 ya no es mayor que 3, la condición se vuelve **falsa** y Arduino **deja** de ejecutar ese código.

Ejemplo

El paraguas. Es como decir: *si llueve, abro el paraguas*. Cuando deja de llover, puedo cerrarlo. La acción de abrir el paraguas la decido justamente con un `if`: "si llueve", y entre las llaves el código que abre el paraguas. En otra parte puedo escribir "si **no** llueve" (la condición **contraria**), y entre las llaves el código que cierra el paraguas. Igual: *si tengo hambre, como*. Todas las decisiones de la máquina se construyen así.

1.2 Variables que dependen del exterior

Hasta ahora, en nuestros programas, las variables `a` y `b` las **decidimos nosotros** al principio, y siempre quedan iguales: la comparación da siempre el mismo resultado. En realidad podemos hacer que una variable dependa de algo que **pasa afuera** de la placa, por ejemplo de un **pulsador** (*pulsante*) apretado o no.

2 El pulsador como entrada, el LED como salida

Conectamos a Arduino un **pulsador**. El pulsador es un dispositivo que **manda una información** a Arduino: la información va **hacia la placa**. Entonces la patita (*piedino*) donde lo conectamos debe ser una **entrada** (*ingresso*, `INPUT`). Puede ser una patita **cualquiera** (la 3, la 9, la 12, la que quieran): lo importante es configurarla como `INPUT`. En el ejemplo usamos la **patita 7**.

Del otro lado conectamos un **LED**, con su resistor, a una patita configurada como **salida** (*uscita*, `OUTPUT`): en el ejemplo la **patita 3**. Después el LED vuelve a **GND**.

Nota

El símbolo del LED. El LED es un **diodo** que emite luz: su símbolo es el del diodo (un **triángulo** con una **barrita**) con dos **flechitas** que salen, la luz.

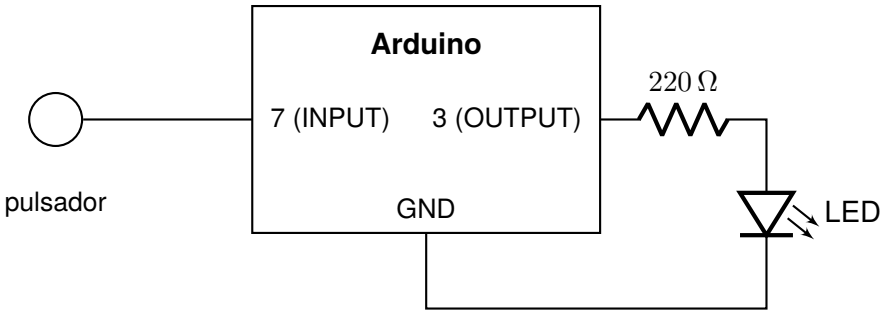


Figura 1: El esquema dibujado en la pizarra: el pulsador manda la información a la patita 7 (entrada), la patita 3 (salida) enciende el LED, que vuelve a GND. La conexión completa del pulsador se ve en el circuito de Tinkercad.

2.1 El pulsador se vuelve a controlar en cada vuelta

Arduino no “siente” el pulsador de forma continua. El código dentro de `loop` se **repite todo el tiempo**: en **cada repetición** Arduino tiene que **volver a controlar** si el pulsador está apretado o no.

Para leerlo está la orden que ya vimos: `digitalWrite` sirve para **escribir**, `digitalRead` (con la **R mayúscula**) sirve para **leer**. Entre paréntesis le pasamos el número de la patita, aquí el 7:

```
a = digitalRead(7);
```

El valor leído (`HIGH` o `LOW`) lo guardamos en una variable, por ejemplo `a`. Después nos preguntamos: si `a` está **alto**, enciendo el LED; si `a` está **bajo**, lo apago.

3 El programa completo

Este es el programa escrito en la pizarra.

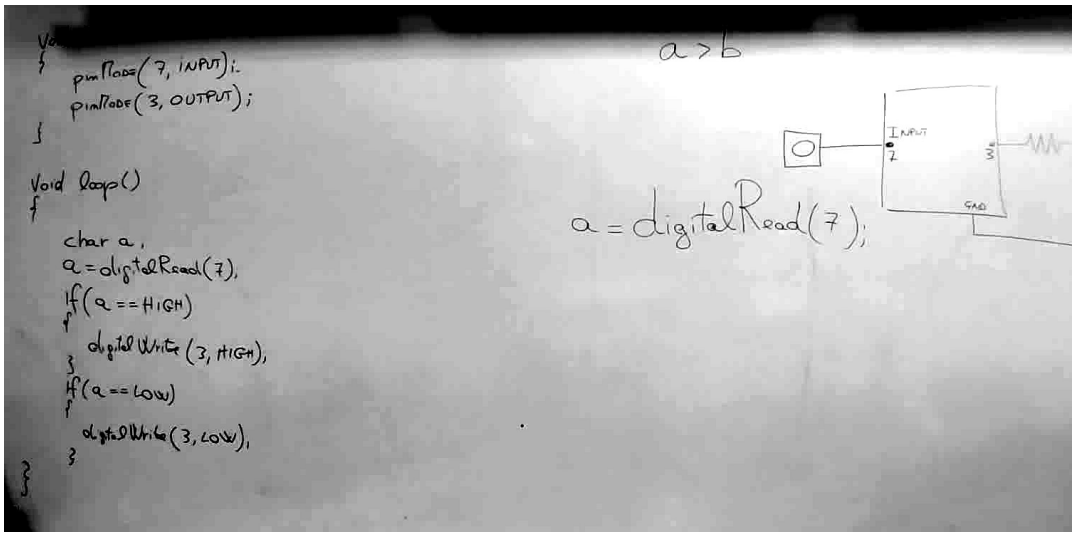


Figura 2: La pizarra: a la izquierda el programa, a la derecha el esquema con el pulsador en la 7 y el LED en la 3.

```
void setup()
{
  pinMode(7, INPUT);
  pinMode(3, OUTPUT);
}

void loop()
{
  char a;
  a = digitalRead(7);

  if (a == HIGH)
  {
    digitalWrite(3, HIGH);
  }

  if (a == LOW)
  {
    digitalWrite(3, LOW);
  }
}
```

- En `setup`: la patita **7** es una **entrada** (está el pulsador), la patita **3** es una **salida** (está el LED).
- En `loop`: creamos la variable `char a` y guardamos en ella el **estado del pulsador**, leído con `digitalRead(7)`. Con estas dos líneas creamos una variable y leemos el pulsador.
- Primer `if`: si `a` está **alto** (pulsador apretado), `digitalWrite(3, HIGH)` **enciende** el LED.
- Segundo `if`: si `a` está **bajo** (`LOW`), `digitalWrite(3, LOW)` lo **apaga**. Cuando sacamos el dedo del pulsador, `digitalRead` devuelve `LOW`, entonces también `a` se vuelve bajo y el LED se apaga.

Dentro de cada par de llaves aquí hay **una sola instrucción**, pero podríamos poner **muchas**: todas se ejecutan si la condición es verdadera.

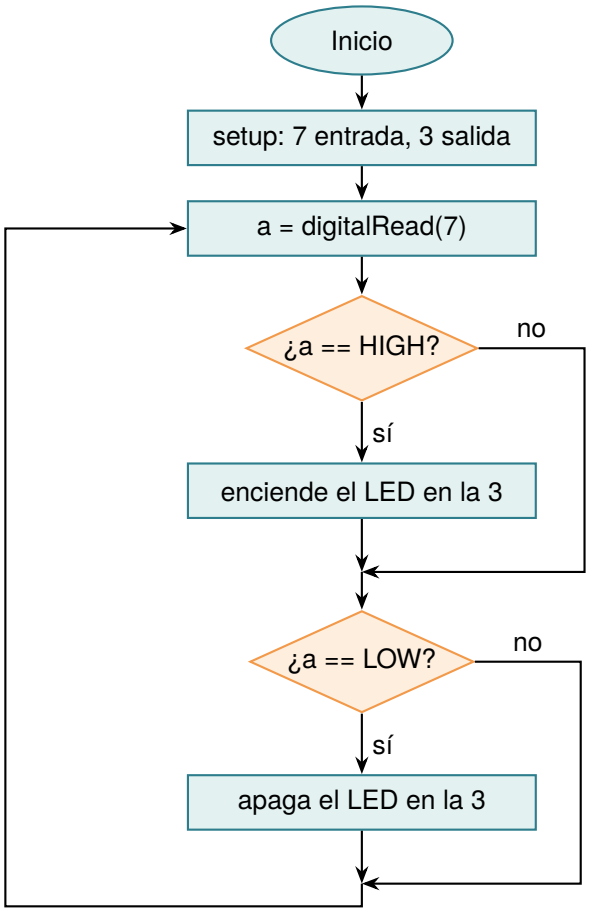


Figura 3: El programa del pulsador: en cada vuelta de `loop` Arduino lee la patita 7 y decide si encender o apagar el LED. El `loop` no termina nunca: después del último control se vuelve a empezar desde la lectura.

4 Lo probamos en Tinkercad

4.1 El LED

1. Necesitamos un **Arduino** y una **protoboard** (*breadboard*).
2. Conectamos **enseguida GND y 5 V** a las líneas de alimentación (*rail*) de abajo de la protoboard (negro al menos, rojo al más), así la plataforma ya está lista: estos cables se **ponen siempre**.
3. En el **lado derecho** de la protoboard unimos las líneas de abajo con las **de arriba** con dos cables: **negro** entre los dos menos, **rojo** entre los dos más. Así también las líneas de arriba tienen **GND y 5 V**: el pulsador va a tomar los 5 V justamente de la línea de arriba.
4. Ponemos un **LED** conectado a la **patita 3** de Arduino, como ya hicieron. Como siempre el **resistor** (*resistore*) es de **220 Ω**, y el cátodo del LED va a la línea de **GND**.

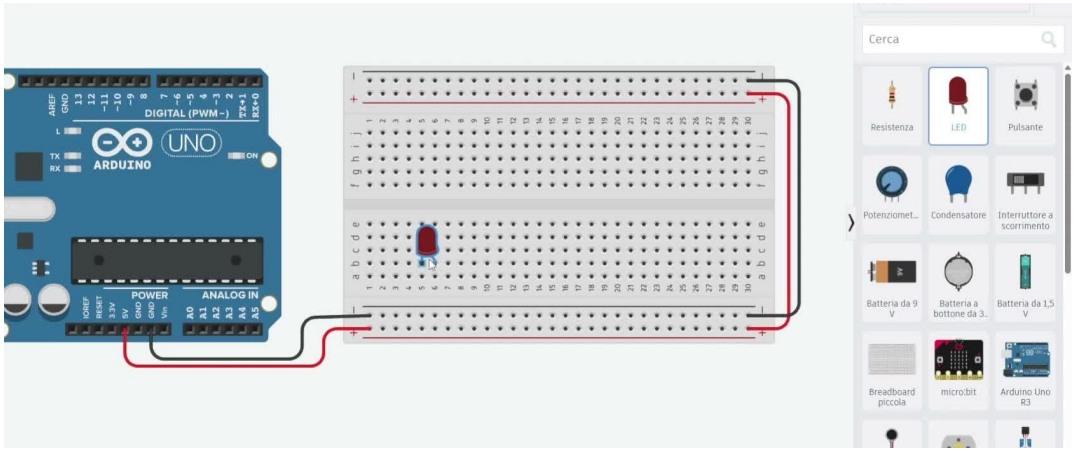


Figura 4: La plataforma lista: GND y 5 V de Arduino en las líneas de abajo, y en el lado derecho dos cables (negro y rojo) que llevan GND y 5 V también a las líneas de arriba.

4.2 El pulsador

El pulsador se conecta así (**cómo** funciona esta conexión lo vemos en **electrónica**, el lunes):

1. se pone el **pulsador** a caballo del **canal central** de la protoboard (“aquí en el medio”);
2. desde una patita del pulsador se pone un **resistor** hacia la línea de **GND**: lo pueden dejar de **1 kΩ**, como lo propone Tinkercad;
3. un **cable rojo** conecta el **otro lado** del pulsador (el lado derecho) a los **5 V**;
4. entre el resistor y el pulsador quedan **dos agujeros libres**: eligen **uno**, el que quieran, y desde ahí llevan un cable a la **patita 7**. Atención: pongan el cable **justo en el agujero**, en el centro.

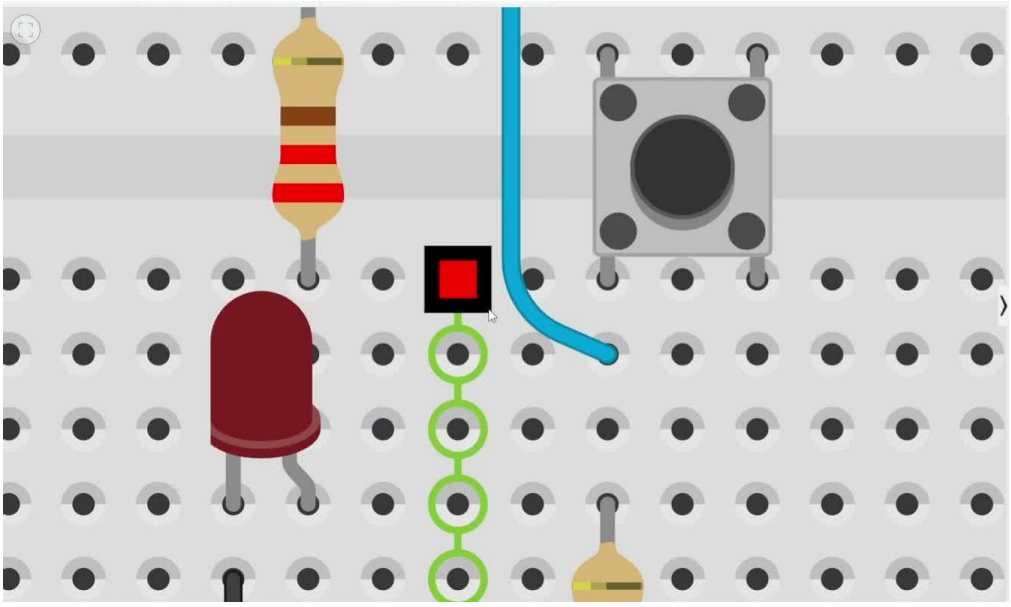


Figura 5: El pulsador a caballo del canal central; abajo, el resistor hacia GND; el cable celeste va a uno de los agujeros libres entre pulsador y resistor.

Nota

Para qué sirve el resistor del pulsador. Cuando el pulsador **no está apretado**, la patita 7 está conectada a **GND** a través del resistor, entonces `digitalRead(7)` lee **LOW**. Cuando lo **aprietan**, el pulsador conecta la patita 7 a los **5 V** y la lectura se vuelve **HIGH**. Sin el resistor, con el pulsador suelto la patita no estaría conectada a nada y la lectura sería **casual**. Un resistor usado así se llama **resistor de pull-down**: lo vemos mejor en electrónica.

4.3 El código

Se abre el panel **Código** (*Codice*): en un proyecto nuevo empieza en modo **Bloques** (*Blocchi*), entonces se pasa al modo **Texto** (*Testo*). Ahí Tinkercad pone un **programa de ejemplo** (hace parpadear el LED de la placa, con `LED_BUILTIN` y `delay(1000)`): se **borra** todo y se **copia** el programa de la pizarra. Cuando terminen, inicien la **simulación**: al apretar el pulsador el LED se tiene que **encender**, al sacar el dedo se tiene que **apagar**.

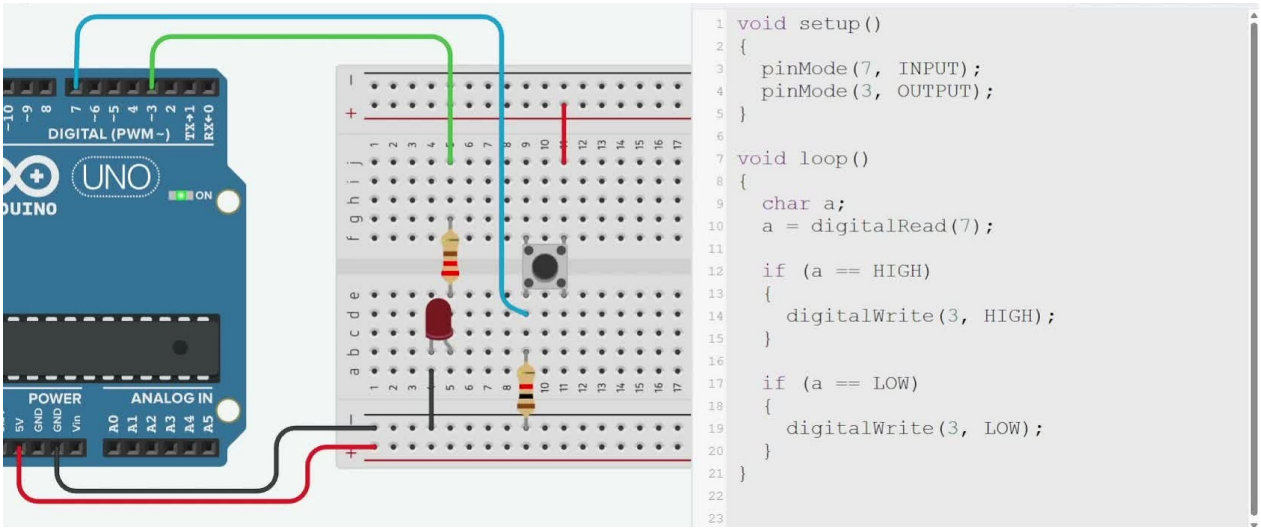


Figura 6: El circuito terminado en Tinkercad: cable verde de la patita 3 al resistor de 220 Ω y al LED, cable celeste de la patita 7 al pulsador, resistor de 1 kΩ hacia GND, cable rojo del pulsador a los 5 V. A la derecha el código.

Atención

Las llaves en el teclado italiano. La llave **abierta** { se hace con **AltGr + Mayúscula + è**; la llave **cerrada** } con **AltGr + Mayúscula + +**. Si una tecla del teclado no funciona se avisa; mientras tanto se puede **copiar** una llave ya escrita (por ejemplo la de la línea 6) y **pegarla** donde hace falta.

5 Los errores más frecuentes

Cuando el programa no funciona, antes que nada se **lee el mensaje** que Tinkercad escribe **abajo**, en rojo: dice **qué** falta y **en qué línea**.

- **Mayúsculas y minúsculas**. El C las distingue: `setup` se escribe todo en **minúscula** (no `Setup`), `pinMode` con la **p minúscula** y la **M mayúscula** (no `PinMode`), `INPUT` y `OUTPUT` todo en **mayúscula**, `digitalRead` con la **R mayúscula**.
- **Nombre mal escrito**. Si falta una letra (por ejemplo `digitaRead`), el compilador no conoce la orden: "`'digitaRead' was not declared in this scope`", es decir "no fue declarado", y propone la alternativa correcta ("`suggested alternative: 'digitalRead'`").
- **Falta el punto y coma** (*punto e virgola*). El mensaje es "`expected ';' before '}'`": "esperaba un punto y coma antes de la llave cerrada". El número de línea dice dónde mirar: el punto y coma falta al final de la instrucción **antes** de esa llave.
- **Llaves**. Cada llave abierta debe tener su llave **cerrada**; si no las cierran todas, el programa no se puede probar. Atención también a la llave "torcida" (paréntesis o corchete en lugar de la llave) y a la llave de más que no cierra nada: el mensaje es del tipo "`expected '}' at end of input`".
- **Circuito y código deben coincidir**. Si el LED está conectado a la patita **2** pero en el código está `digitalWrite(3, ...)`, el LED no se enciende: el número en el código debe ser **el real**. Lo mismo si un cable termina en la **columna equivocada** de la protoboard (por ejemplo la 6 en lugar de la 5).
- **Los cables de alimentación** de la protoboard (GND y 5 V) se ponen **siempre**: sin ellos, nada funciona. Si el circuito parece correcto pero al apretar el pulsador el LED no se enciende, controlen el **lado derecho**: sin los dos cables que unen las líneas de abajo con las de arriba, la línea de donde el pulsador toma los 5 V **no tiene alimentación**.
- **Nada de cables sueltos**: se trabaja **siempre en la protoboard**.
- Si movieron por error la vista o un componente, el botón con la **flecha hacia la izquierda** arriba a la izquierda (**deshacer**) vuelve un paso atrás; después se **reinicia** la simulación.

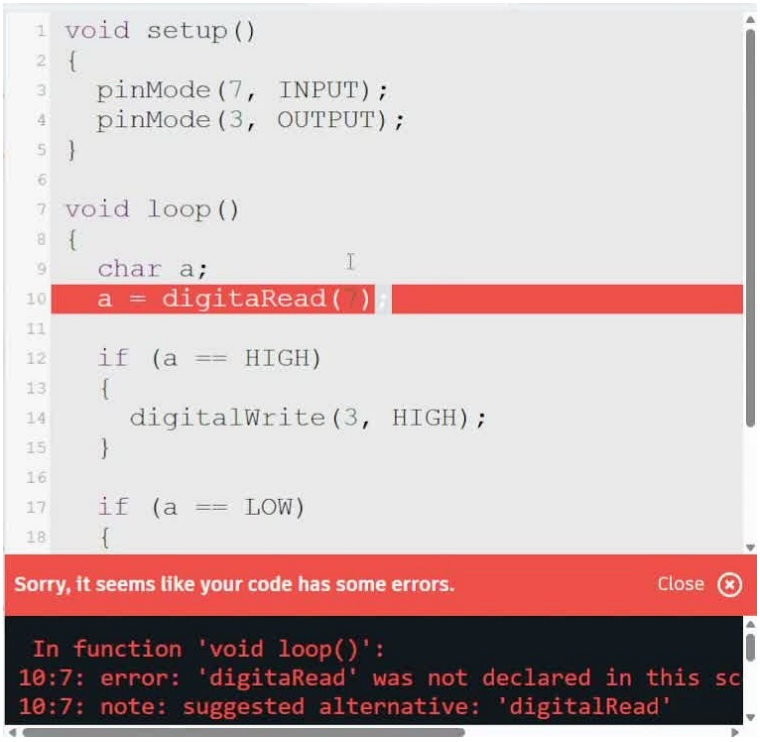


Figura 7: Un nombre mal escrito: la línea 10 se pone roja y abajo aparece el mensaje con el número de línea y la alternativa sugerida.

6 Ejercicio: cuatro LED y un pulsador

Atención

Primero una copia de seguridad. El proyecto del pulsador que funciona **guárdenlo**: es su **referencia** para volver a ver cómo se escribe el código. Para el ejercicio creen un **proyecto nuevo**. Y antes de pasar al ejercicio, el primer circuito tiene que **funcionar**.

Consigna. Creen un Arduino con **cuatro LED** conectados, los cuatro que vuelven a **GND**, en las patitas que ustedes quieran. Después conecten un **pulsador** de la misma manera que antes.

- Si el pulsador está **apretado**, se encienden **solamente dos** LED (elegidos por ustedes).
- Si el pulsador **no está apretado**, se encienden **solamente los otros dos**.
- En los dos casos tienen que prever también el **apagado** de los otros dos, para estar seguros de que se apaguen.

Bastan **dos if**, no más.

6.1 Una solución

En la solución hecha en clase los LED están en las patitas **2** (rojo), **3** (verde), **4** (rojo) y **5** (verde); el pulsador está en la patita **8**. Pulsador apretado: se encienden los **dos rojos** y se apagan los verdes. Pulsador suelto: al revés.

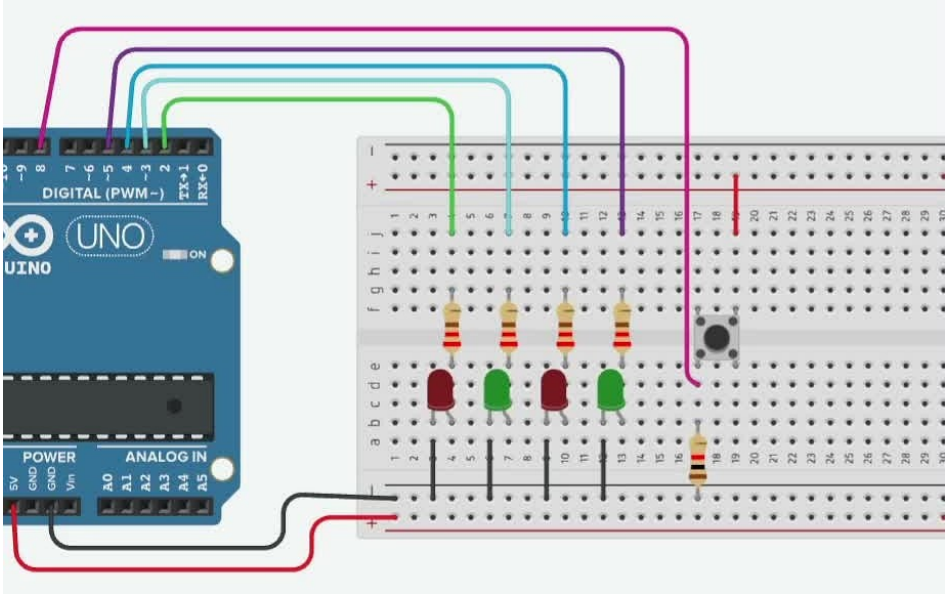


Figura 8: Cuatro LED en las patitas 2, 3, 4, 5 (rojo, verde, rojo, verde), cada uno con su resistor de 220 Ω; el pulsador en la patita 8 (cable rosa), conectado como antes.

```
void setup()
{
  pinMode(2, OUTPUT);
  pinMode(3, OUTPUT);
  pinMode(4, OUTPUT);
  pinMode(5, OUTPUT);
  pinMode(8, INPUT);
}

void loop()
{
  char a;
  a = digitalRead(8);

  if (a == HIGH)
  {
    digitalWrite(2, HIGH);
    digitalWrite(3, LOW);
    digitalWrite(4, HIGH);
    digitalWrite(5, LOW);
  }

  if (a == LOW)
  {
    digitalWrite(2, LOW);
    digitalWrite(3, HIGH);
    digitalWrite(4, LOW);
    digitalWrite(5, HIGH);
  }
}
```

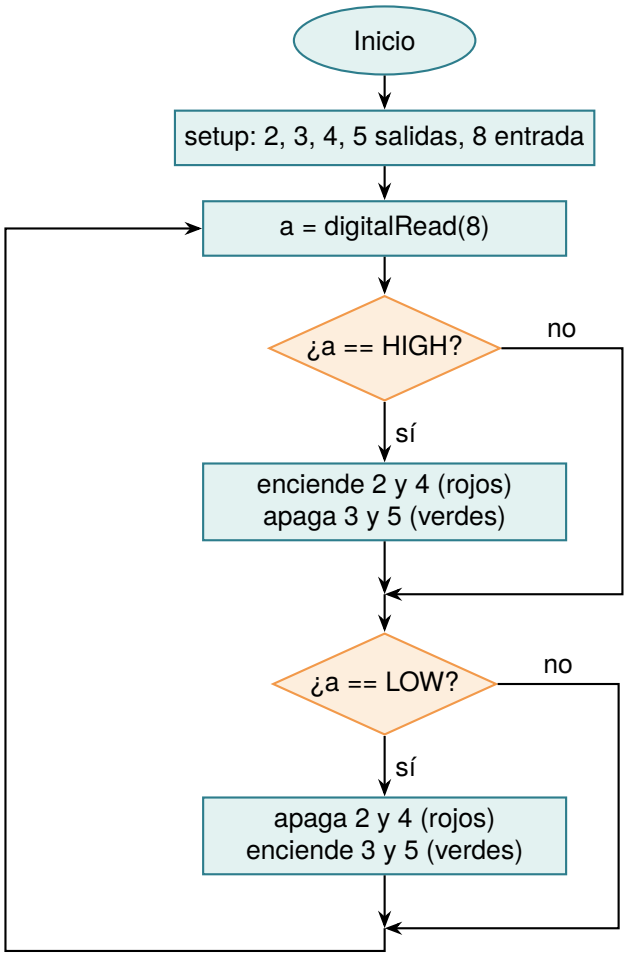


Figura 9: El ejercicio de los cuatro LED: dos if, uno para el pulsador apretado y uno para el pulsador suelto; en cada uno se encienden dos LED y se apagan los otros dos.

Nota

Por qué apagar siempre los otros dos. Un `digitalWrite(..., HIGH)` deja el LED encendido hasta que alguien escribe `LOW`. Si en el primer if encendemos los rojos pero no apagamos los verdes, los verdes que quedaron encendidos de la vuelta anterior **siguen encendidos**: por eso en cada if se escriben **los cuatro** LED.

7 Qué recordar de esta lección

- `if (condición) { ... }`: si la condición es **verdadera** ejecuta el bloque entre las llaves, si es **falsa** lo **salta**.
- Una variable puede depender del **exterior**: `a = digitalRead(7);` lee un **pulsador**.
- La patita del pulsador es un **INPUT**, la del LED un **OUTPUT**.
- El `loop` se repite: en **cada vuelta** el pulsador se **vuelve a leer**.
- El pulsador en Tinkercad: a caballo del canal, **resistor de 1 kΩ** hacia GND, **5 V** del otro lado, **cable a la patita** entre pulsador y resistor; las líneas de arriba **unidas** a las de abajo.
- Cuando algo no anda, se **lee el mensaje** de abajo: dice qué falta y en qué línea. Atención a

mayúsculas, **punto y coma**, **llaves** y a los **números de las patitas**.

- Llaves en el teclado italiano: **AltGr + Mayúscula + è** y **AltGr + Mayúscula + +**.
- Con cuatro LED: **dos if**, y en cada uno se decide el estado de **todos** los LED.

8 Glosario: palabras en italiano que se usan en clase

- **pulsante**: pulsador.
- **premuto, rilasciato**: apretado, suelto.
- **piedino**: patita, pin.
- **ingresso, uscita**: entrada, salida.
- **condizione**: condición.
- **vero, falso**: verdadero, falso.
- **graffa / graffe**: llave / llaves.
- **parentesi tonde**: paréntesis.
- **punto e virgola**: punto y coma.
- **resistore**: resistor.
- **filo / fili**: cable / cables.
- **breadboard**: protoboard.
- **rail**: línea de alimentación de la protoboard.
- **lato destro**: lado derecho.
- **leggere, scrivere**: leer, escribir.
- **alto, basso**: alto, bajo.
- **accendere, spegnere**: encender, apagar.
- **lavagna**: pizarra.
- **Codice, Blocchi, Testo**: Código, Bloques, Texto (en Tinkercad).
- **maiuscolo, minuscolo**: mayúscula, minúscula.
- **errore**: error.
- **esercizio**: ejercicio.
- **dispensa**: apunte.